

CONSULTORIA ESPECIALIZADA EM CIÊNCIA DE DADOS 1

Validação hold-out e Pipeline

Aula 05 · 01/09/2026

O curso, semana a semana (1/2)

Aula (Dia/Mês)	Assunto	Evento
1 — 04/08	Introdução: fundamentos da avaliação de modelos	
2 — 11/08	Métricas de classificação I (confusão, precisão, recall, F1)	Lançamento do projeto
3 — 18/08	Métricas de classificação II (ROC-AUC, PR, log-loss)	
4 — 25/08	Métricas de regressão (MAE, RMSE, R^2 , resíduos)	Projeto: proposta
5 — 01/09	Validação hold-out e Pipeline	
6 — 08/09	Validação cruzada I (k-fold, estratificado)	
7 — 15/09	Validação cruzada II (grupos, temporal, aninhada)	
8 — 22/09	Busca exaustiva de hiperparâmetros (grid search)	Projeto: dados
9 — 29/09	Busca aleatória e menção a AutoML	

O curso, semana a semana (2/2)

Aula (Dia/Mês)	Assunto	Evento
10 — 06/10	Curvas de validação	Desafio 3 (subst.)
11 — 13/10	Curvas de aprendizado	
12 — 20/10	Seleção e comparação de modelos	
13 — 27/10	Limiar de decisão e calibração	
14 — 03/11	Target Leakage e reprodutibilidade	
15 — 10/11	Seleção de atributos	
16 — 17/11	Estudo de caso integrado	
17 — 24/11	Comunicação e reprodutibilidade dos resultados	Desafio 6 (subst.)
18 — 01/12	Revisão integrada e apresentações	Apresentação final

Objetivo da aula

Separar os dados em **treino, validação e teste** para estimar o desempenho com honestidade — e usar **Pipeline** para não vazar informação.

PARTE 1

Treino, validação e teste

Uma fatia para aprender, uma para escolher, uma para o juízo final.

1

Três conjuntos, três papéis

Um corte para o teste; do resto, um corte para a validação



Separe uma fatia **antes de treinar** e a mantenha escondida; meça o modelo **só nela**.

- **Treino** = a lista de exercícios — o modelo aprende aqui (usa muitas vezes).
- **Validação** = o simulado — você testa jeitos de estudar e escolhe o melhor (várias vezes).
- **Teste** = a prova oficial — a nota que vale, honesta — uma única vez.

Como obter os três: dois cortes encadeados

```
from sklearn.model_selection import train_test_split

# 1) separe o TESTE (vai para o cofre)
X_resto, X_test, y_resto, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)
# 2) do resto, separe TREINO e VALIDACAO
X_tr, X_val, y_tr, y_val = train_test_split(
    X_resto, y_resto, test_size=0.25, stratify=y_resto, random_state=0)
```

Estratificar e fixar a semente

- **stratify=y** — mantém a proporção das classes em cada conjunto — essencial com classe rara.
- **random_state** — fixa o sorteio: o mesmo split toda vez, para o experimento ser reproduzível.
- **Confira depois** — verifique que as proporções bateram nos três conjuntos antes de seguir.

O fluxo correto

- **1. Treine vários modelos** — no conjunto de treino.
- **2. Compare na validação** — escolha o campeão e ajuste os hiperparâmetros aqui.
- **3. Confirme uma vez no teste** — rode o campeão no teste — esse é o número que você reporta.
- **A analogia** — o teste é o exame final lacrado, aberto só no dia da prova.

PARTE 2

Vazamento no pré-processamento

Padronizar ou imputar com a base inteira contamina o teste.

2

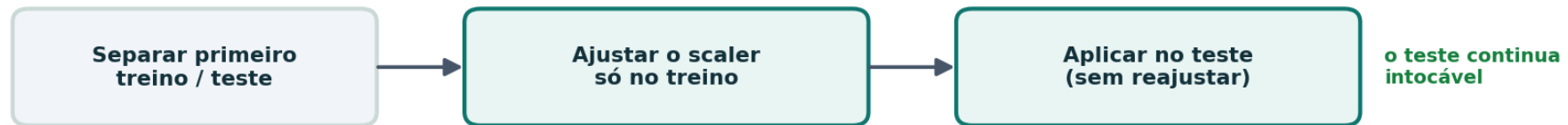
O vazamento sutil do pré-processamento

Vazamento sutil: onde o scaler é ajustado muda tudo

✗ Errado



✓ Certo



Ajustar o scaler (ou imputador, ou seletor de atributos) na base inteira faz o treino 'enxergar' o teste.

A regra de ouro e onde o vazamento se esconde

Aprenda as transformações **só no treino**; aplique-as ao teste **sem reajustar**.

- **Padronização / normalização** — a média e o desvio devem vir só do treino.
- **Imputação de faltantes** — a estatística de preenchimento também.
- **Seleção de atributos** — escolher colunas olhando o dado todo já é vazamento.

fit × transform e o vazamento

O erro clássico: dar **fit no dataset inteiro** (treino + teste) antes de separar.

```
# ERRADO — o scaler 'vê' o teste ao calcular média/desvio:
sc = StandardScaler().fit(X)          # usou treino + teste!

# CERTO — fit só no treino:
sc = StandardScaler().fit(X_train)
X_test_ok = sc.transform(X_test)      # teste só recebe transform
```

fit = aprender; transform = aplicar. O teste nunca entra no fit — senão a avaliação fica otimista (vazamento).

PARTE 3

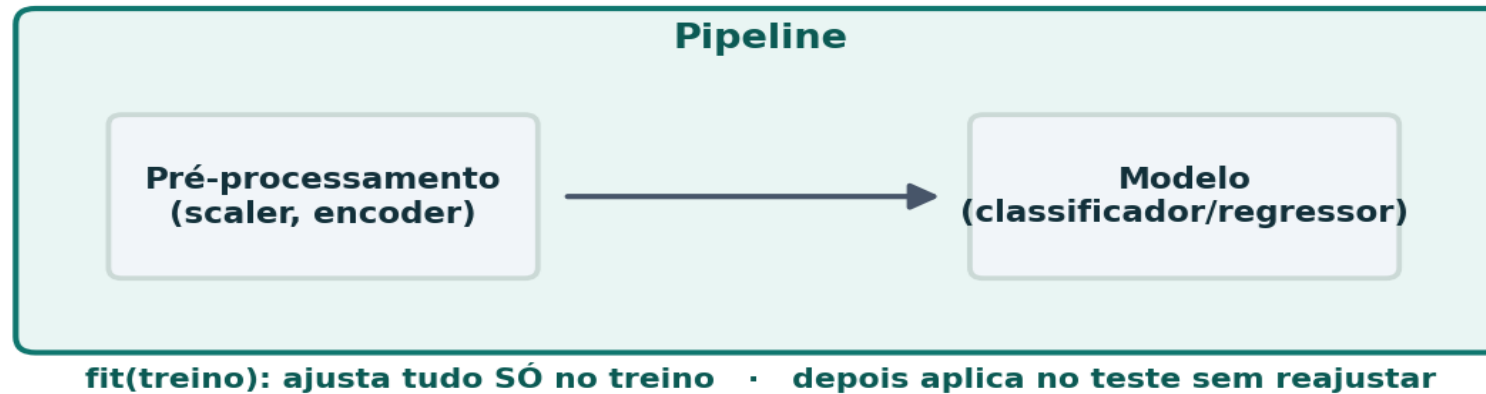
A solução: o Pipeline

Encadeia pré-processamento e modelo e aprende tudo só no treino.

3

O Pipeline

O Pipeline encadeia as etapas e aprende as transformações só no treino



Ao chamar fit no Pipeline, ele ajusta o scaler/encoder apenas no treino; predict aplica as mesmas transformações ao teste.

ColumnTransformer: cada tipo de coluna

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

prep = ColumnTransformer([
    ("num", StandardScaler(),  colunas_numericas),
    ("cat", OneHotEncoder(),   colunas_categoricas),
])
```

Padroniza as numéricas e codifica as categóricas — cada grupo com seu tratamento, em um só objeto.

Pipeline: pré-processamento + modelo

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

modelo = Pipeline([
    ("prep", prep),                # o ColumnTransformer acima
    ("clf", LogisticRegression()),
])
modelo.fit(X_tr, y_tr)           # ajusta scaler/encoder SO no treino
modelo.predict(X_test)           # aplica as mesmas transformacoes
```

Um único objeto faz o pré-processamento e a previsão — juntos e na ordem certa.

Por que o Pipeline evita o vazamento

- **fit só no treino** — o Pipeline ajusta as transformações apenas com os dados de treino.
- **Aplica sem reajustar** — no teste (e em produção) ele usa as estatísticas já aprendidas.
- **Seguro na validação cruzada** — refaz o pré-processamento a cada fold, automaticamente (Aula 6).

Erros comuns e boas práticas

Erros comuns

- Padronizar/imputar antes de separar treino e teste.
- Escolher hiperparâmetros olhando o teste.
- Medir várias vezes no teste.
- Fazer o pré-processamento fora do Pipeline.

Boas práticas

- Separe o teste antes de qualquer coisa.
- Coloque todo o pré-processamento no Pipeline.
- Decida na validação; teste só no fim.
- Estratifique e fixe a semente.

PARTE 4

Além da sala: hold-out e pipelines no mundo real

4

Produção: skew e drift

Terminado o teste, com tudo validado, o modelo entra em **produção** (uso geral). Aí surgem dois problemas, em **momentos diferentes**:

Skew — no deploy (AGORA)

- **O que é** — treino e produção tratam os dados de formas diferentes (ex.: esqueceram de padronizar).
- **Impacto** — ótimo no teste, ruim em produção — falha silenciosa.
- **Defesa** — o MESMO Pipeline no treino e em produção.

Drift — com o tempo (DEPOIS)

- **O que é** — os dados mudam com o tempo e saem da distribuição do treino.
- **Impacto** — o modelo era bom e vai piorando.
- **Defesa** — monitorar os dados novos e re-treinar.

Skew = problema de espaço (treino $\neq$ produção); drift = problema de tempo (o mundo muda).

Hold-out no tempo: teste é o futuro

Com data, o corte é por **tempo**: treina no **passado**, testa no **futuro** — nunca sorteie.

- **Validar o modelo** — segure os últimos períodos conhecidos como teste (ex.: treina 2000–2023, prevê 2024–2025) e compare com o real.
- **Na hora de valer** — re-treine com TODOS os dados até o corte; a separação serviu só para medir.

Os períodos passados ficam no treino (ensinam a sazonalidade); só o trecho de teste, depois do corte, sai.

Global holdout: validar o PRODUTO, não o modelo

Mudança de assunto: aqui **não validamos um modelo** — medimos se o **produto melhorou de verdade** (algo que vocês farão na carreira).

- **A ideia** — 1–5% dos usuários ficam na versão antiga por meses; o resto recebe as novidades.
- **Mede** — comparando os dois grupos num indicador de negócio (receita por usuário, retenção...).
- **A/B × global holdout** — o A/B testa UMA mudança; o global holdout mede o efeito de TUDO, no longo prazo.
- **Exemplo** — em 3 meses, receita/usuário: R\$ 42 (antigo) × R\$ 45 (novidades) = +7%. Se supera o ruído e nada piora, valeu.

Mesmo nome 'hold-out', outro uso: aqui é para avaliar o NEGÓCIO, não treinar um modelo.

Um corte só é jogar a moeda

Avaliar com uma **única divisão treino/teste** depende da sorte do sorteio — a nota **balança conforme o corte**.

- **O risco** — um corte 'fácil' infla a nota; um 'difícil' a derruba — nenhum é o desempenho verdadeiro.
- **A ideia melhor** — dividir os dados em várias partes, treinar e testar em cada uma e tirar a MÉDIA.
- **Resultado** — uma estimativa estável, que não depende de um sorteio único.

É a validação cruzada — tema da próxima aula (Aula 06).

